

# FUSE-NF

## Next Steps for NESS and Beyond

Binding-preserving semantic normalization,  
useful abstractions, and executable concept learning

---

Prepared for the FUSE-NF and NESS teams

September 30, 2026

### Central recommendation

Develop FUSE-NF into a system that discovers and validates reusable semantic abstractions, rather than judging progress mainly by how much a parsed graph can be simplified. Preserve entity identity, event identity, variable sharing, uncertainty, and provenance. Let downstream inference and prediction determine which abstractions deserve to be retained.

**Two complementary outcomes.** Independently of NESS, this should provide a more reliable semantic interface for querying, retrieval, reasoning, and program search. Within NESS, it should provide stable relation vocabulary and reusable binding computations whose contribution can be measured against strong same-information controls.

**Status.** This is a proposed research and engineering programme, not a report of new experiments. It distinguishes the supplied concept paper and NESS records, limited inspection of repository exports, and recommendations that remain to be tested.

## Executive summary

The most promising next step is to connect FUSE-NF’s existing consolidation methods to a more demanding notion of usefulness. Frequent subtrees, similar role-filler distributions, mutual information, paraphrase alignments, and autoencoder features are all plausible ways to *propose* a shared representation. None, by itself, establishes that two meanings are interchangeable or that a new meta-node will help a reasoning or predictive system.

The FUSE-NF concept paper explicitly treats these five approaches as alternatives that may be combined [1, Sec. 4.3]. The recommendation here is to keep that diversity, but give all methods a common candidate format and a common evaluation process. A candidate should state what it preserves, what it abstracts away, how its variables bind, what evidence supports it, and which downstream task or computational saving justifies its adoption.

**The first priority is reliable structure, not maximal compression.** Use a small typed semantic substrate, modular logical templates, and explicit scopes. Separate exact normalization from reversible packaging, approximate abstraction, and background inference. Retain the source graph. In particular, do not erase directional roles, negate-away inconvenient distinctions, or identify entities merely because their embeddings are similar.

**The second priority is shared-variable structure.** A useful concept should often be an executable relation pattern with an explicit argument interface. A bundle such as “a helpful person holds a key that opens the container containing the target” is interesting because its variables must refer to the *same* helper, key, and container. The NESS reference examples make this same-witness requirement explicit [2, slides 8–10, B7]. FUSE-NF should expose and test such patterns, including existing join-capable candidates where available, rather than relying only on a tree-restricted export.

**The third priority is evaluation that separates invariance from insensitivity.** Paraphrases should preserve the relevant representation, while role reversals, negation, modality, event-identity changes, and broken shared bindings should remain distinguishable. A constant representation would be perfectly invariant and completely useless. Accordingly, invariance and counterfactual sensitivity must be measured together, alongside task performance and cost.

**For NESS, the first integration should be narrow.** Feed canonical primitive facts and a small bank of validated, feature-valued binding programs into the existing symbolic interface. Keep neural access to the same observations in all controls. Start with verified bindings and a fixed representation, then introduce noisy extraction and uncertainty. Do not make reduced dependence on engineered extraction a prerequisite for showing predictive value with good bindings.

**The principal learning loop should be conditional admission.** Generate a candidate abstraction, check its semantic contract, fit a matched downstream consumer, and test whether it improves held-out prediction or delivers a measured computational saving. Recompute residual errors after admissions. A NESS-specific concept should be admitted for its contribution beyond the current bank and its simpler components, not because it was frequent in the mining corpus.

**A realistic first deliverable** is a small, reproducible benchmark and candidate registry: exact normalization tests, paraphrase/contrast families, several reusable binding motifs, a general-purpose query or missing-relation task, and one controlled NESS integration. More elaborate graph autoencoders, jointly learned parsers, and continual ontology adaptation can follow after that experiment identifies where the remaining bottleneck lies.

## Contents

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>Executive summary</b>                                                     | <b>1</b>  |
| <b>1 Starting point, evidence, and intended change</b>                       | <b>3</b>  |
| <b>2 Separate four operations that should not share one guarantee</b>        | <b>4</b>  |
| <b>3 A small typed substrate and modular logical templates</b>               | <b>5</b>  |
| <b>4 Treat parser disagreement as evidence, not calibrated truth</b>         | <b>6</b>  |
| <b>5 Strengthen the five existing proposal methods</b>                       | <b>7</b>  |
| <b>6 Use multiple normalized views with explicit refinement information</b>  | <b>9</b>  |
| <b>7 Choose abstractions by conditional utility and semantic constraints</b> | <b>10</b> |
| <b>8 Worked example: learn a reusable helper-access abstraction</b>          | <b>11</b> |
| <b>9 NESS integration: a semantic interface and a concept-learning loop</b>  | <b>12</b> |
| <b>10 Experiments that can make the next result decisive</b>                 | <b>14</b> |
| <b>11 Implementation roadmap and acceptance gates</b>                        | <b>17</b> |
| <b>12 What to defer, and what the resulting contribution could be</b>        | <b>18</b> |
| <b>A Suggested interface and candidate records</b>                           | <b>19</b> |
| <b>B Implementation tests and a minimal admission loop</b>                   | <b>21</b> |
| <b>References and source notes</b>                                           | <b>23</b> |

### Reading guide

Sections 1–5 address the general FUSE-NF programme. Sections 6–7 give the proposed representation and learning criteria. Sections 8–9 explain the NESS connection. Sections 10–11 give experiments and milestones. The appendices collect interface fields, implementation tests, and an admission-loop sketch.

# 1 Starting point, evidence, and intended change

## 1.1 What the source materials establish

The original FUSE-NF paper proposes a progression from syntactic and logical parses to a normalized semantic graph, statistical consolidation, background PLN inference, and possible graph-transformer integration. Its five mining methods occupy Section 4.3; the paper itself describes the design as a rough concept rather than a completed implementation [1, Secs. 1, 4–5].

The accompanying developer update reports a corpus of roughly 2,300 parsed sentences, some plausible synonym consolidation and multi-atom compression, and continuing verification of the paraphrase and autoencoder methods. It also reports parse variation across repeated LLM calls and suggests replacing an expanding prompt with reviewable logical templates [4]. These are useful observations from the developer, not independent performance measurements in this note.

A limited inspection of the linked exports provides several concrete qualifications:

- The subtree export distinguishes a full inventory of 7,216 patterns from a faithful view containing 2,125 rooted-subtree units and 931 multi-atom proposals, on 2,352 records. Its header excludes joins from that view, while explicitly noting that the broader inventory includes joins and lifted patterns. The immediate recommendation is therefore to *inspect and expose the existing join-capable inventory*, not assume it must all be implemented anew [5].
- The role-filler export already distinguishes lexical corroboration from actual role-change proposals and labels proposed mappings as provisional. This is a useful separation to preserve when adding stronger semantic tests [6].
- The MI export distinguishes genuine combinations from containment-based pairs and records shared-variable alignments. Its header reports 207 passing pairs, of which 149 are containment-based. Thus, proposal counts should not be interpreted as counts of independently useful new concepts [7].

These statements concern readable headers and selected entries, not a code audit or an experimental reproduction. The autoencoder export was linked but its contents were not available through the inspected web preview; no implementation-level assessment of it is asserted here [8]. Neither the generators nor the downstream validation machinery were executed for this note. A reproducible experiment should first pin a commit and archive its inputs and configuration.

## 1.2 A stronger target than a visually cleaner graph

The suggested research question is:

Can a system discover reusable semantic abstractions from imperfect parses, preserve the distinctions required for reasoning, and make useful computation more accessible to bounded downstream learners?

This changes the success criterion without discarding the existing work. A frequent frame may be valuable because it shortens a query, improves retrieval across paraphrases, or reduces symbolic search effort. A larger binding composition may be valuable because it changes a NESS prediction. An approximate semantic class may be valuable for one task while being too coarse for another.

The normalizer should therefore record *which kind of benefit is claimed*. Compression, semantic equivalence, predictive improvement, and computational acceleration are different claims and need different tests.

## 2 Separate four operations that should not share one guarantee

The concept paper aims at a unique, compact, semantics-preserving graph and then introduces fuzzy consolidation of nearby meanings [1, Secs. 2–4]. The recommendation below intentionally narrows the exact guarantee and makes the fuzzy stage less destructive. It is a proposed revision of that design, not a claim that the paper already specifies these distinctions.

| Operation               | Intended effect                                                      | Appropriate contract                                                               |
|-------------------------|----------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Exact normalization     | Remove representational variation inside a declared formal fragment. | Preserve the interpreted input graph’s semantics under specified rules.            |
| Reversible packaging    | Name and cache a recurring subgraph or computation.                  | Recover its expansion and all required witness bindings.                           |
| Approximate abstraction | Share a coarser description across related meanings.                 | Declare discarded distinctions, applicability conditions, and tested distortion.   |
| Background inference    | Add conclusions supported by facts and rules.                        | Keep premises, rule provenance, context, and uncertainty separate from extraction. |

Table 1: Four operations with different correctness obligations.

### 2.1 Exact normalization should have a bounded, explicit scope

A practical exact layer can normalize bound-variable names, canonicalize permitted argument encodings, sort order-insensitive conjunctions, and remove redundant duplicates where the chosen semantics permits this. It must preserve negation scope, quantifiers, modality, role direction, and event and entity identity.

The target should be a deterministic representative relative to a declared rule set, not an unqualified globally minimal representation of natural-language meaning. Termination and confluence require separate justification for the chosen rewrites; neither alone establishes global minimality. In particular, a formal rewrite can preserve the meaning of an incorrect parse perfectly. Parser fidelity remains a separate measurement.

Begin with a deliberately restricted, executable fragment and clearly mark unsupported constructions. A fragment that passes strong tests is more useful than a broad formalism with informal guarantees.

### 2.2 Packaging must not hide the witnesses that later programs need

For a pattern with public variables  $\mathbf{u}$  and internal variables  $\mathbf{v}$ , one might introduce

$$M(\mathbf{u}) := \exists \mathbf{v} \bigwedge_i a_i(\mathbf{u}, \mathbf{v}). \quad (1)$$

This is a useful query definition, but the Boolean fact  $M(\mathbf{u})$  does not by itself retain the identities or multiplicity of the internal witnesses. To call the transformation reversible, keep the matching source atoms and substitutions, or a lossless descriptor that recovers them.

A symbolic consumer can then request either a Boolean test, a set of matching tuples, or a packed feature. Those interfaces should not be conflated. A motif that forgets a helper identity cannot later support a test that the same helper is trusted.

## 2.3 Approximate abstraction should add a view, not erase evidence

The original example merges Agent and CoAgent into Participants [1, Secs. 4.4, 6.4]. For some broad descriptions, participant membership is a useful *additional* feature. Replacing the original roles, however, can remove who acted and who accompanied the action. Directional relations make the problem even clearer: sender and recipient are not interchangeable simply because both are people.

Likewise, a shared Transfer view may help generalize across “gave,” “handed,” and “passed,” but it should not assert that all details of those events are identical. Preserve lexical manner, ownership versus temporary possession, and other task-relevant residuals. Approximate similarity need not be transitive; a chain of pairwise similarities should not silently collapse into a global equivalence class.

## 2.4 Do not turn normalization into unmarked world-model inference

The original design includes PLN enrichment and causal labels [1, Secs. 4.5–4.6]. Retain that possibility, but distinguish what the text states from what the system infers. “A occurred before B” is not, by itself, evidence that A caused B or was a prerequisite of B. An inferred completion, possession change, or causal link should name its rule and supporting context.

This matters for evaluation: an abstraction should not appear predictively valuable merely because it was supplied with unreported extra knowledge. It also matters for maintenance: retracting a premise must invalidate dependent conclusions without deleting the original observation.

# 3 A small typed substrate and modular logical templates

The suggestion to break the parser prompt into logical templates is well aligned with the next phase. The aim should be a small compositional language plus reviewable template modules, not another monolithic ontology written in a different format.

## 3.1 Core representation

A minimal substrate should distinguish entities, events or states, predicates, argument roles, and scope-bearing contexts. A schematic vocabulary is:

```
Entity(x, type)
Event(e, event_type)
Role(e, role_name, filler)
Relation(predicate, argument_1, argument_2, ...)
InContext(statement_id, context_id)
Temporal(event_1, relation, event_2)
CoreferenceHypothesis(mention_1, mention_2)
```

Negation, modality, quotation, and quantification need explicit scoped structure, not merely optional labels attached to whichever node is convenient. A statement within “Alice believes ...” must not automatically become an asserted world fact. Unknown attachment or unresolved reference should remain representable.

The exact names above are illustrative. Where the repository already has suitable types and conventions, retain them and document the mapping. The necessary change is the contract, not a gratuitous vocabulary migration.

For each fact, retain source spans, an observation identifier, parser/template versions, its scope, and an availability boundary. Keep mention identity distinct from resolved entity identity. Two occurrences of “Alex” are not necessarily one entity, and two occurrences of “a key” are not necessarily the same key.

### 3.2 Templates as checked constructors

A transfer template could construct a frame with an event, sender, recipient, and object. Its specification should include positive examples, role-reversal negatives, scoped or negated cases, optional fields, and examples where the template must abstain.

The parsing workflow can be staged: detect a likely construction, select a template, fill typed roles, validate the resulting graph, and preserve alternatives when multiple analyses remain possible. Mechanical validators should catch malformed structures; they should not hallucinate missing semantics in order to make the structure pass.

For example, the template for “X gave Y Z” should not also be responsible for inventing whether Y retained Z later. A separate inference rule may model that expectation, with its own evidence and defeasibility.

### 3.3 Make parser uncertainty actionable

Audit a small set of repeated parses before expanding the prompt. Classify disagreements as lexical choice, argument role, attachment, coreference, event decomposition, scope, or unsupported assertion. Then attach each failure to a template or representation requirement.

This gives a concrete improvement loop: fix one template, rerun its targeted cases, and check that unrelated templates did not regress. Keep a fallback representation for constructions not yet covered. A parser forced to choose an incorrect known frame can be worse than one that emits a partial but faithful graph.

## 4 Treat parser disagreement as evidence, not calibrated truth

Repeated parsing is potentially valuable, but a frequently repeated parse is not necessarily correct. Repeated calls to the same model and prompt can share systematic errors. Therefore record agreement statistics separately from any calibrated probability of semantic correctness.

### 4.1 A bounded distribution over graph hypotheses

A useful experimental representation is

$$Q_t(G) = \sum_{k=1}^K w_{k,t} \mathbf{1}[G = G_{k,t}], \quad w_{k,t} \geq 0, \quad \sum_k w_{k,t} = 1. \quad (2)$$

Here  $G_{k,t}$  are compatible, prefix-available graph hypotheses. The weights may begin as heuristic ensemble weights. They become probabilities of correctness only to the extent that a defined calibration experiment supports that interpretation. When the retained hypotheses have incomplete coverage, record that limitation and provide an abstention or unresolved state rather than pretending the finite set is exhaustive.

For an initial diagnostic, use three to five parses on a selected subset, not twenty parses of every sentence. Add diversity through template choices or an independent extraction view only where that helps locate a

real error source. The point is to discover which ambiguities matter, not maximize the number of model calls.

## 4.2 Preserve joint alternatives: do not construct an impossible hybrid

Suppose two equally weighted hypotheses are

```
G1: helps(Bela, Alex), holds(Chen, key1)
G2: helps(Chen, Alex), holds(Bela, key1)
```

In neither hypothesis does the same person both help Alex and hold the key. Taking the union of facts creates a false shared-witness match. Multiplying independently estimated edge confidences can make the same mistake in probabilistic form.

For a Boolean binding program  $\pi$ , a safer bounded approximation is

$$u_{\pi}(t) = \sum_k w_{k,t} \mathbf{1}[\text{execute}(\pi, G_{k,t}) = 1]. \quad (3)$$

Execution occurs within each compatible hypothesis before aggregation. More scalable factored representations are possible later, but they must retain the dependencies relevant to joins. An implementation should also distinguish “false in a complete local world” from “not established in an incomplete extracted graph.”

## 4.3 Keep evidence strength, semantic truth, and predictive reliability separate

Three quantities may differ: confidence that a parse matches the text, confidence that an asserted proposition is true of the world, and reliability of a program as a predictor in a particular context. The NESS consumer can learn the third without proving the first two.

Paraphrases, repeated parses, and several overlapping macros may all trace back to one observation. Their provenance should prevent them from being counted as independent confirming facts. Calibration should be tested on reviewed examples and reported with coverage, including the cases on which the parser abstains.

# 5 Strengthen the five existing proposal methods

The five methods in the concept paper should remain recognizable. The proposed change is to treat their outputs as candidates with different evidence, not interchangeable certificates of semantic equivalence [1, Sec. 4.3].

## 5.1 Frequent subtree mining: retain the baseline, add binding-aware views

Frequency is useful for finding reusable fragments and choosing what to inspect. Continue reporting support and coverage, but distinguish independent sentence families from repetitions, paraphrases, or duplicate parses of the same source.

Alongside the tree-restricted view, expose bounded connected relational patterns with shared variables. Start by auditing what the existing broader miner already supplies. Candidate growth can be constrained by atom count, number of free variables, relation types, and estimated execution cost. A small initial range, such as two to five atoms, is an experimental choice rather than a universal limit.

Add typed anti-unification: generalize several concrete patterns into a common template while preserving repeated-variable constraints. For example, two instances of “a helper holds the key that opens the relevant container” should yield a common argument structure, not just a bag of predicate names. Test its matches under entity renaming and deliberately broken identity links.

A shape-only abstraction and a lexicalized one can coexist. The shape may generalize across predicates but also overgeneralize; retaining both lets downstream evidence decide which level is useful.

## 5.2 Role-filler clustering: propose correspondences, not unconditional merges

Similar fillers are evidence that roles occupy a related semantic region. They are not evidence that the roles are interchangeable. Buyers and sellers may both be organizations; actors and patients may both be people. Conversely, corresponding roles can have different observed distributions because the corpus samples different situations.

Represent a candidate as a *typed, contextual mapping*, possibly with an argument permutation, rather than a global role rename. For a proposed correspondence between buying and selling, identify which role is buyer, which is seller, and what event-level assumptions align the two descriptions. Preserve the mapping’s direction and applicability conditions.

Require tests that swap roles, introduce a third entity, negate the event, or change a relevant modifier. Cases that the candidate cannot distinguish should remain separate. The existing export’s distinction between lexical evidence and a role-change proposal is a good starting point [6].

## 5.3 Mutual-information grouping: discount structural and dataset artifacts

A high association score can arise because one pattern contains another, because two units always occur in a generated template, or because several records are near-duplicates. High mutual information also need not mean positive co-occurrence: a strong exclusion relation can carry information without justifying a conjunctive pack.

Require positive co-occurrence when that is the intended hypothesis, inspect within-event versus document-level association, and retain the variable alignment that makes a proposed conjunction executable. Continue separating containment-based pairs from genuinely new combinations, as the current export already does [7].

Next, test whether a candidate predicts an additional relation or outcome beyond its components and the current representation. Conditional MI can be a useful conceptual target, but direct estimation with sparse data and rich neural context is difficult. A held-out residual-prediction probe is often a simpler operational test. Report uncertainty across independent families rather than treating every pairwise scan as an independent discovery.

## 5.4 Paraphrase alignment: learn invariance and its boundaries together

Use paraphrase sets paired with meaning-changing near-neighbours. A useful family might include active/passive variants, a lexical paraphrase, a role reversal, and a negated or modal version. Scope and reference must be checked, not assumed to remain fixed merely because the text is similar.

Align graphs with typed role constraints and explicit entity/event correspondences. Distinguish equivalence from one-way entailment and from broad relatedness. “Alice gave Bob a book” and “Bob received a book” need not be equivalent: the latter does not identify the giver.

Avoid evaluating the normalizer solely with paraphrase labels produced by the same parser that supplied its inputs. A small independently reviewed subset and controlled semantic constructions provide a stronger diagnostic. Keep all variants of a source family in the same data split.

## 5.5 Autoencoder analysis: retain a baseline, then test structural objectives

The concept paper proposes a shallow autoencoder over graph-feature counts and interprets tied encoder weights as a source of consolidation candidates [1, Sec. 4.3.5]. That is a reasonable baseline for detecting statistical regularities, but tied weights do not establish substitutability, and a representation that discards binding topology cannot be expected to recover it reliably.

A proposed extension is a structural encoder with objectives for typed edges, role assignments, and shared-variable constraints. Add contrastive training that keeps verified paraphrases close while separating role and binding counterfactuals. A predictive bottleneck can then encourage representations that support missing-relation, next-event, or continuation prediction.

Use auxiliary structural and contrastive losses rather than prediction alone: a predictor may ignore a rare but crucial distinction. Any discovered latent group still needs an explicit symbolic interpretation and the same admission tests as candidates from the other methods. A large new autoencoder should not delay the first useful end-to-end experiment.

## 6 Use multiple normalized views with explicit refinement information

For NESS and many other consumers, one irreversible final graph is unnecessarily restrictive. Retain a canonical primitive graph and a set of views:

$$\text{observations} \longrightarrow \text{primitive facts} \longrightarrow \text{frames and motifs} \longrightarrow \text{task-specific abstractions.} \quad (4)$$

Each view should identify its source facts, expansion or query definition, public argument interface, residual distinctions, and validation status.

For “John handed Mary the book,” a Transfer view can expose sender, recipient, object, and event. A more abstract social-exchange view may be useful for predicting a response, but should not replace the lower representation or be asserted without a declared rule. A physical-prediction task might retain the manner of transfer; a pronoun-resolution task might need discourse and mention structure.

It is tempting to call this a normalization lattice. For implementation, a *versioned directed acyclic graph of abstraction dependencies* is the safer requirement. A mathematical lattice would additionally require suitable meet and join operations, which are not supplied merely by having several levels of abstraction. Exact equivalences and approximate links should be stored as different relations.

For a lossless package, require a decoder and residual record satisfying

$$\text{decode}(N(G), R(G)) \equiv_R G, \quad (5)$$

where  $\equiv_R$  is equivalence in the declared exact rewrite fragment. For a deliberately lossy view, specify which queries or distinctions it is intended to preserve and measure failures on those tests.

A multi-view design also avoids unnecessary global disagreement. Two tasks can use different abstractions of the same event without requiring the system to decide that one is the only correct semantic normal form.

## 7 Choose abstractions by conditional utility and semantic constraints

### 7.1 A common admission objective

Let  $S$  be the currently available candidate bank,  $C$  a proposed addition, and  $\hat{\theta}_S$  the parameters of a downstream consumer fitted with bank  $S$ . For a held-out admission set, define

$$\Delta(C | S) = \mathcal{L}_{\text{admit}}(S, \hat{\theta}_S) - \mathcal{L}_{\text{admit}}(S \cup \{C\}, \hat{\theta}_{S \cup \{C\}}). \quad (6)$$

The consumers must be trained under matched protocols. Merely adding an untrained feature and observing no gain says little about its usefulness. Conversely, giving only the candidate condition more parameters, training steps, or observations confounds the result.

A proposed score is

$$J(C | S) = \Delta(C | S) - \lambda \text{Complexity}(C | S) - \mu \text{ExtraCost}(C | S). \quad (7)$$

Complexity should include the stored definition and any new fitted tables or parameters. Cost should include extraction, matching, and consumption rather than only the final network call. Set the weights and accounting units before admission; also report the uncombined accuracy, memory, and latency results so that the trade-off remains interpretable.

Predictive gain is subject to semantic gates: scope preservation, binding preservation, required counterfactual sensitivity, acceptable calibration, and bounded harm on protected distinctions. Good aggregate loss cannot license a representation that silently turns “did not give” into “gave.”

### 7.2 Not every useful normalization must improve prediction

Exact canonicalization may be justified by reproducibility and equivalence checking. A reversible macro may be justified by fewer query operations or a smaller search space at unchanged answers. A general-purpose retrieval view may improve query matching without helping next-token loss. These should remain legitimate forms of progress.

The special requirement for a *NESS predictive concept* is that it earn a predictive role, or demonstrably deliver the same predictive result more cheaply. A transparent implementation should give candidates separate fields for semantic status, compression benefit, computational benefit, and predictive benefit.

### 7.3 Measure benefit beyond simpler components

If  $C$  packages patterns  $C_1$  and  $C_2$ , compare it against a consumer that already receives  $C_1$  and  $C_2$ , not against one deprived of their facts. When the point is shared identity, include an ablation with the same local relations but without the shared-witness constraint.

For a learned concept bank, repeat selection after refitting the consumer and refreshing residual errors. This follows the conditional-utility principle in the NESS programme: refit, refresh residuals, and keep an equal-budget no-growth control [2, slides 12, 22]. Do not interpret a visually deeper hierarchy as intrinsically more intelligent or useful.

### 7.4 Prevent discovery from consuming the test set

Use grouped partitions for mining, consumer fitting, candidate admission, and final testing. All paraphrases, repeated parses, and controlled variants of a source episode should remain within one group.

Additional splits should hold out lexical variants or compositions where those forms of transfer are the target.

A practical separation is  $D_{\text{mine}}$ ,  $D_{\text{fit}}$ ,  $D_{\text{admit}}$ , and  $D_{\text{test}}$ . Calibration can use a designated part of the fitting data. Repeatedly searching the same admission set can still overfit; use a limited search budget, nested selection or rotating admission folds, and a final untouched test. Report paired uncertainty across independent episodes and training seeds, not only across correlated tokens.

## 7.5 The expected advantage is computational, not new information

A deterministic normalized feature of the observed history does not add information to a hypothetical unbounded predictor that already has that history. It may nevertheless expose a computation that a finite neural consumer would not learn or execute efficiently. NESS explicitly motivates its symbolic path in these terms and requires composition controls to receive the same observations [2, slide 2].

This distinction strengthens the research claim. The question is whether a representation makes a useful computation accessible at a reasonable cost, not whether renaming existing facts somehow creates new evidence.

## 8 Worked example: learn a reusable helper-access abstraction

The following constructed example adapts the kind of shared-witness test used in the NESS slides; it is not a new experimental result or a replay of their saved world [2, slides 8–10].

Suppose a controlled story establishes that Alex trusts Bela, Bela helps Alex, Bela holds a key, the key opens a container, and the sought object is inside that container. Extraction yields:

```
trusts(Alex, Bela)
helps(Bela, Alex)
holds(Bela, key1)
opens(key1, box1)
inside(object1, box1)
```

The description might be paraphrased using “is holding” rather than “holds,” or by a passive construction. FUSE-NF should preserve the same local binding structure while keeping any additional meaning that the paraphrase genuinely introduces.

### 8.1 A concept with a useful public interface

Define the candidate query

$$\text{HelperAccess}(A, H, O) := \text{helps}(H, A) \wedge \exists K, C [\text{holds}(H, K) \wedge \text{opens}(K, C) \wedge \text{inside}(O, C)]. \quad (8)$$

Its public variables include  $H$ . This matters because the next program is

$$\text{TrustedAccess}(A, O) := \exists H [\text{trusts}(A, H) \wedge \text{HelperAccess}(A, H, O)]. \quad (9)$$

If `HelperAccess` returned only “someone can help,” it would discard the identity required for the trust test. Store matching  $(H, K, C)$  tuples or source pointers when later programs may need them.

## 8.2 A counterfactual that breaks the join without removing local evidence

Add an unrelated actor Dana and another person Chen. In world  $W_1$ , use the trust edges

```
trusts(Alex, Bela)
trusts(Dana, Chen)
```

In world  $W_2$ , swap their endpoints:

```
trusts(Alex, Chen)
trusts(Dana, Bela)
```

Keep the helping, holding, opening, and containment facts fixed. The number of trust edges and their endpoint degree counts are unchanged. Alex still trusts someone; someone still helps Alex and holds the relevant key. But only  $W_1$  supplies one person satisfying both requirements in Equation (9).

A candidate normalizer should preserve that distinction. A bag of relation labels, an unconditional participant merge, or a union of incompatible parse alternatives can lose it.

## 8.3 What is learned, and what is not

FUSE-NF can learn a common lexical/frame view and propose HelperAccess from recurring, aligned patterns. NESS can search for its composition with trust and learn how the output affects continuation probabilities. Whether Alex actually asks for help is not a logical consequence of the graph; it is an outcome distribution learned by the consumer.

The evaluation should hold out names, wording, distractors, and selected compositions. It should compare the abstraction against its primitive expansion and its simpler components. A hard-coded task rule that directly maps TrustedAccess to the correct continuation would test a supplied solution, not learned predictive use.

## 8.4 Avoid over-interpreting a concept name

The same discipline applies to other candidate macros. A pattern that says “finished task T before event E” may be a useful temporal abstraction, but it should not be named CompletedPrerequisite unless a prerequisite relation is independently established. Give a new node a descriptive name tied to its executable definition; treat more ambitious interpretations as additional hypotheses.

# 9 NESS integration: a semantic interface and a concept-learning loop

## 9.1 Match the actual symbolic contracts

The supplied NESS reference distinguishes probability-valued readers from feature-valued binding programs. It also shows a direct binding-feature path to the final correction head, rather than forcing every program output through the reader mixture [2, slides 3–4].

The broader NESS motivation also connects semantic relation extraction to programs using symbolic history [3, 16:42–21:50]. The first FUSE-NF integration should respect the technical distinction above:

- A **binding program** returns a Boolean, a witness set, a structured feature, or uncertainty-aware support. Its predictive interpretation is learned by the consumer.
- A **probability-valued reader** must return a normalized forecast over the prediction vocabulary. It needs fitted outcome statistics or a learned predictive model; a semantic confidence is not such a forecast.

For the first pilot, use the feature-valued path. Leave the existing probability-reader bank and mixture interface intact. A later reader can use a normalized semantic frame as its key, fit outcome counts only after outcomes are observed, and back off when support is sparse, following the reader-learning distinction in the NESS slides [2, slides 5–6].

## 9.2 Proposed data flow

Let  $H_t$  be observed history,  $h_t$  the adaptive neural representation, and  $B_t$  the available semantic record. A minimal extension is

$$B_t = \text{FUSE}(H_t; \text{schema}, \text{parser}, \text{rule versions}), \quad (10)$$

$$f_{\pi,t} = \text{execute}(\pi, B_t), \quad (11)$$

$$m_t(y) = \sum_j w_{j,t} q_{j,t}(y), \quad (12)$$

$$p_t(y) = \text{softmax}_y[\log m_t(y) + \delta_{0,t}(y) + \delta_{\Pi,t}(y; f_t)]. \quad (13)$$

The final two expressions follow the form of the supplied NESS reference [2, slides 3–4, B9]; the FUSE interface is the proposed addition. The schema should carry typed arguments, witnesses or witness references, scope, availability, uncertainty diagnostics, and provenance. The adapter need not consume all metadata as neural features, but execution and auditing must retain it.

Keep raw/canonical facts accessible to a same-information neural graph baseline. If FUSE contributes inferred knowledge or additional memory, expose the same knowledge to the comparison condition when claiming a compositional advantage.

## 9.3 Enforce prefix causality at the parser boundary

For next-token prediction, every supplied fact must be derivable from information available before the predicted token. Parsing an entire sentence and attaching its graph to earlier token positions leaks future information, even when the symbolic program itself only performs a valid query.

Start with completed sentences or event segments whose facts become available only after their observed boundary. Later, incremental parsing can add and revise hypotheses. Store `available_from` separately from the time described by an event: learning today about yesterday’s event does not make that fact available to yesterday’s predictor.

Availability is not the same as present validity. A previously observed holding relation may have ceased to hold after a later transfer. Keep event-scoped assertions and state-validity intervals distinct; a state query should not join mutually incompatible times merely because all of its facts were observed.

A required automated test is *prefix invariance*: change or append an unseen suffix while keeping  $H_t$  fixed, and verify that  $B_t$ , all retrieved memories, program outputs, and the prediction are unchanged. Training outcomes may update parameters or counts after observation; they must not enter the current observation key.

## 9.4 Learn the vocabulary and the program bank on different schedules

A stable first integration should fix normalization rules and discovered concepts while fitting the NESS consumer. This isolates whether the representation is useful. Then add an outer loop that proposes a new abstraction, validates it, updates the program search vocabulary, refits the consumer, and evaluates conditional gain.

Only after that loop works should parser, abstraction bank, program search, and neural cap all adapt together. Otherwise a failed run leaves too many plausible explanations: poor extraction, unstable identities, redundant concepts, ineffective search, or an incapable consumer.

This staged path is compatible with the NESS distinction between reliable-binding experiments and a separate extraction-uncertainty track [2, slide 23]. Engineered extraction remains a legitimate tool in the reliable-binding branch.

## 9.5 Use uncertainty without pretending the consumer is linear

Equation (3) gives one possible uncertainty-aware feature. Another is to supply the distribution of witness sets, support, disagreement, and unresolved mass. A more expensive reference method is to make a complete prediction for each graph hypothesis and mix the normalized forecasts:

$$p_t(y) = \sum_k w_{k,t} p_t(y \mid G_{k,t}). \quad (14)$$

For a nonlinear consumer this generally differs from predicting once using average features. Compare the two on small cases before choosing the cheaper approximation. All alternatives must use the same observed prefix, and weights must be described honestly as calibrated or heuristic.

## 9.6 Protect the interface during continual learning

Normalization changes can break old symbolic programs without deleting any program from memory. Use stable concept identifiers with immutable definitions, explicit versions, dependency records, and migration adapters. A new definition should not silently replace an old one under the same identifier.

Retain old-domain query and predictive tests when adding a new domain or revising templates. Measure old-task performance *before* return-domain training, not only how fast it recovers afterward. The NESS materials explicitly distinguish recovery, internal forgetting, and retained functional use [2, slides 19–22]. FUSE-NF could help interface stability, but this note does not claim that normalization solves catastrophic forgetting.

# 10 Experiments that can make the next result decisive

## 10.1 A small diagnostic benchmark before a much larger corpus

Begin with a manageable set of independently reviewed semantic families. As a planning example, 100–200 families with several paraphrases and meaning-changing variants can expose many structural failures. Repeat parsing only on an ambiguity-enriched subset. These counts are workload suggestions, not a statistical power claim; determine inferential precision from the measured variance and the number of independent families.

Include role reversal, negation, modal scope, quantifier scope, coreference, distinct versus shared events, temporary possession versus ownership, and shared-witness counterfactuals. Include partial and genuinely ambiguous inputs on which abstention or several hypotheses is appropriate.

Generate controlled examples from known small worlds where useful, but complement them with reviewed natural text. Keep world construction and outcome generation separate from the parser. Store the latent world for evaluation only unless the experiment is explicitly the verified-input condition.

## 10.2 General-purpose FUSE-NF evaluation

| Question                                           | Primary measurement                                                    | Essential control                                                            |
|----------------------------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Does exact cleanup preserve meaning?               | Query agreement, scope tests, alpha-renaming invariance, idempotence.  | Primitive source graph in the declared formal fragment.                      |
| Do paraphrases align without collapsing contrasts? | Paraphrase consistency and contrast sensitivity reported separately.   | Lexical matching and embedding-only views on the same inputs.                |
| Do abstractions support inference?                 | Missing-relation prediction or held-out query answering.               | Components alone; same supplied facts; hidden target removed from all views. |
| Does packaging reduce computation?                 | Query/search time, explored candidates, memory, and unchanged answers. | Expanded definitions and a matched caching baseline.                         |
| Does uncertainty help?                             | Task performance, calibration, and coverage as input noise increases.  | Single parse, hard consensus, and verified-input reference.                  |

Table 2: Independent uses of FUSE-NF need not depend on a NESS training run.

For missing-relation prediction, remove the target from every representation and from any inference path that directly reveals it. For compression, count the dictionary, residual facts, and stored witnesses, not only the number of displayed meta-nodes. For semantic retrieval, require precision as well as recall; broad concepts can increase retrieval while introducing false matches.

These tests can establish useful outcomes on their own: more stable knowledge-base ingestion, shared queries across lexical variants, faster relational matching, and reusable program primitives. They should not be described as achieved benefits until measured.

## 10.3 First NESS experiment: verified inputs and hard interfaces

Hold the backbone, observations, trainable capacity, and training schedule as closely matched as possible. Use one continuation task where a shared binding matters. Report residual differences in compute rather than claiming exact cost matching when it is not achieved.

For exact macros, also compare named execution with fully expanded execution. They should return identical results. Any advantage may then come from search efficiency, caching, or optimization rather than new semantics. For approximate abstractions, use the semantic contrast suite to quantify what was lost.

Evaluate complete continuations, not only a convenient decision token. Also report ordinary-text loss where the task mixes structured episodes with prose. Use held-out names, paraphrases, distractor configurations, and compositions. Include a matched no-growth control so that candidate admission is not simply rewarded for another round of training.

## 10.4 Second NESS experiment: extraction uncertainty

With the program bank and consumer design fixed, compare a single parse, hard consensus, a dependency-preserving uncertain interface, and a verified-input reference. The reference should provide only semantic

| Condition | Input/computation                                                                            | Purpose                                                        |
|-----------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| N0        | Neural consumer with the same canonical facts; no candidate binding features.                | Same-information baseline.                                     |
| N1        | Existing primitive binding grammar and a fixed search budget.                                | Baseline symbolic access.                                      |
| N2        | N1 plus candidate FUSE motifs as named, typed primitives.                                    | Does the learned vocabulary help prediction or bounded search? |
| N3        | N2's primitive facts and component features, without the crucial shared-witness composition. | Is binding composition doing useful work?                      |
| N4        | N2 with deliberately mismatched binding features, leaving the neural observations correct.   | Does performance depend on world-specific symbolic alignment?  |

Table 3: A focused first comparison, not a request for a full architecture sweep.

information contained in the observed input. It is diagnostic rather than a fair deployment-cost competitor.

This comparison separates extraction damage from the potential value of symbolic computation. If verified graphs help but noisy ones do not, work on the front end. If hard consensus helps as much as the uncertain interface, postpone expensive uncertainty propagation. If neither verified nor extracted structures help, investigate the task, program search, and consumer rather than simply enlarging the parser.

Use the same set of parsed hypotheses for hard-consensus and uncertain conditions where feasible. Charge the cost of generating those hypotheses in deployment comparisons.

## 10.5 Third experiment: conditional concept learning and transfer

After fixed-bank utility is established, allow FUSE-NF to propose abstractions against current residual prediction errors. Compare a frozen bank, frequency-selected growth, and conditional-utility growth at matched search and fitting budgets. Record every proposed candidate, rejection reason, semantic test result, and measured contribution.

Then move to a second vocabulary or domain. Measure whether old programs still match, whether new concepts improve the new task, and whether old predictive use survives. Freeze the schema in an initial transfer experiment; make schema adaptation a separate intervention.

## 10.6 Decide in advance what counts as success

A strong result would show that an automatically proposed abstraction passes the declared semantic tests, transfers to held-out cases, and either improves a relevant task or delivers a clear cost saving at matched quality. For the NESS claim, it should add predictive value beyond strong same-fact controls and simpler components, with an uncertainty interval that supports the chosen practical threshold.

A useful negative result would locate the bottleneck: extraction noise, lack of genuinely new binding structure, duplication of existing features, insufficient consumer capacity, or computational overhead. The protocol should make those explanations distinguishable rather than producing only one aggregate loss number.

## 11 Implementation roadmap and acceptance gates

The milestones are ordered by dependency; some work can run in parallel. Set performance thresholds before examining final test outcomes. Structural correctness gates can be exact within the supported fragment. Later complexity should not delay the first interpretable result.

### 11.1 M0: Make the current experiment reproducible

**Deliverable.** A pinned repository snapshot, data manifest, explicit record/family identifiers, method configurations, seeds, and a concise explanation of each export. Distinguish rendered proposals, admitted rules, and facts actually loaded by the runtime.

**Acceptance gate.** Re-running the documented steps reproduces candidate identities and statistics, or documents the numerical tolerances and sources of nondeterminism. Every sampled proposal traces to source records. The team can explain which outputs are tree-restricted, join-capable, lexicalized, or type-lifted without relying on filenames alone.

**Why first.** Otherwise a change in corpus, parser, threshold, or export filter can be mistaken for a scientific improvement.

### 11.2 M1: Establish the typed interface and semantic regression suite

**Deliverable.** A small supported semantic fragment, modular templates, exact normalizer, source-preserving graph store, and reviewed paraphrase/contrast families. Add explicit scopes, identity conventions, availability boundaries, and unresolved states.

**Acceptance gate.** Supported exact rewrites pass round-trip/query tests; normalization is idempotent; bound-variable renaming preserves results; role and binding reversals remain distinct; out-of-fragment inputs fail explicitly or remain partial. Prefix-invariance tests show that unseen suffixes cannot alter currently available features.

**Why now.** This prevents candidate discovery from learning over silently inconsistent or target-leaking representations.

### 11.3 M2: Unify candidates and expose useful binding motifs

**Deliverable.** A common candidate registry and adapters for existing miners. Audit the broader pattern inventory before adding new join mining. Add bounded typed anti-unification and a small set of validated paraphrase alignments. Store definitions, public variables, witnesses, provenance, and applicability conditions.

**Acceptance gate.** A candidate can be expanded and executed; canonical duplicates are recognized; shared variables remain shared; required witnesses are recoverable; dependency cycles are rejected. Role correspondences are contextual mappings, not unconditional mergers. A reviewer can inspect both supporting and counterexample cases.

**Why now.** This turns heterogeneous mining outputs into comparable executable hypotheses.

### 11.4 M3: Demonstrate one benefit outside NESS

**Deliverable.** A held-out query, retrieval, missing-relation, or bounded-search experiment using the same facts across conditions. Include an expanded-definition baseline and an independent family-level test split.

**Acceptance gate.** Establish either a reproducible task gain or a measured cost reduction with no unacceptable semantic regression. Report dictionary and residual-storage cost, failure cases, and coverage. If no candidate qualifies, diagnose why rather than promoting the highest-ranked failure.

**Why now.** General FUSE-NF value should not depend on the success of a much larger neural experiment.

## 11.5 M4: Run the narrow NESS integration

**Deliverable.** A feature-valued FUSE-to-NESS adapter, fixed-bank verified-input study, matched controls N0–N4, and complete-continuation evaluation. Keep the first backbone and consumer configuration stable.

**Acceptance gate.** Match semantics against the reference interpreter; preserve normalization of final forecasts; pass causal-availability tests; show whether candidate motifs add value beyond primitives and simpler components. Check sensitivity to corrupted bindings and transfer to held-out names and paraphrases. Include extraction and symbolic-execution overhead in the report.

**Why now.** This directly tests the central NESS hypothesis while keeping the sources of failure limited.

## 11.6 M5: Add uncertainty, adaptive admission, and versioned memory

**Deliverable.** A bounded joint-hypothesis interface, calibration analysis, conditional admission loop, and versioned concept registry with dependency-aware invalidation. Add selected parser or template revisions only after the fixed-bank comparison is understood.

**Acceptance gate.** Uncertainty propagation never fabricates an impossible join; gains are assessed on untouched tests; old programs remain executable or undergo explicit migration; old-domain function is measured before relearning. A new candidate can be rejected or rolled back without corrupting stored observations.

**Why last.** Jointly changing parser, ontology, program bank, and neural consumer would otherwise make the experiment difficult to interpret.

# 12 What to defer, and what the resulting contribution could be

Do not make the first milestone depend on a globally canonical representation of unrestricted language, a very large graph autoencoder, a new graph transformer, or online multi-model parsing at every token. Do not require eliminating engineered extraction before testing whether good semantic bindings help NESS. And do not use an impressive number of mined nodes as a substitute for a task-level result.

The broader conceptual opportunity is to connect three learning problems: FUSE-NF proposes a vocabulary of shared semantic descriptions; symbolic program search composes that vocabulary into executable computations; and a downstream consumer determines which computations are useful. These layers need not use the same optimization algorithm, but they can share evidence about utility and failure.

A defensible initial contribution would be narrower than a universal semantic normal form and more informative than a list of plausible clusters:

A system proposes semantic abstractions from imperfect language parses, preserves their executable binding interfaces, rejects meaning-destroying consolidations through contrast tests, and demonstrates held-out inferential or predictive value at measured cost.

## A Suggested interface and candidate records

The following is a proposed contract, not a description of an existing API. Keep the representation language compatible with the current MeTTa-oriented workflow; JSON can be used for interchange and audit records without becoming the semantic foundation.

### A.1 Minimum fields

| Record component      | Required meaning                                                                                                                                                                                     |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Observation           | Stable ID, source identifier/hash, source spans, parser/template versions, and the boundary at which the observation becomes available.                                                              |
| Entity/event identity | World- or episode-scoped IDs; explicit mention links; unresolved or alternative coreference hypotheses. Canonical structural hashes must not merge distinct real events.                             |
| Fact                  | Predicate, typed ordered arguments, scope/context, polarity, provenance, and assertion status. Missing and denied facts are different states.                                                        |
| Uncertainty           | Weight or score type, calibration status, parser agreement counts, dependency/hypothesis identifiers, and unresolved coverage. No unmarked conversion from agreement frequency to truth probability. |
| Candidate definition  | Canonical typed pattern, public parameters, internal variables, exact or approximate status, applicability conditions, and expansion or query semantics.                                             |
| Execution result      | Return type, value or witness tuples, supporting source references, coverage, and unresolved status when relevant.                                                                                   |
| Candidate evidence    | Mining method, support by independent family, paraphrase/contrast results, semantic regressions, downstream metrics, and rejection reasons.                                                          |
| Lifecycle             | Immutable definition version, dependencies, admission status, training/evaluation split hashes, and migration or deprecation information.                                                            |

## A.2 Illustrative candidate record

The values below illustrate the structure only. The candidate is not marked admitted, and no measured gain or calibrated probability is fabricated.

```
{
  "schema_version": "fuse-ness/0.1",
  "candidate_id": "helper_access:v1",
  "kind": "query_macro",
  "public_arguments": [
    {"name": "A", "type": "Person"},
    {"name": "H", "type": "Person"},
    {"name": "O", "type": "PhysicalObject"}
  ],
  "internal_arguments": [
    {"name": "K", "type": "Key"},
    {"name": "C", "type": "Container"}
  ],
  "body": [
    ["helps", "H", "A"],
    ["holds", "H", "K"],
    ["opens", "K", "C"],
    ["inside", "O", "C"]
  ],
  "scope_policy": "same-compatible-context",
  "return_contract": {
    "value": "matching-public-tuples",
    "retain_internal_witnesses": true,
    "retain_source_fact_ids": true,
    "allow_unresolved": true
  },
  "semantic_status": "exact-query-definition",
  "observation_policy": "prefix-available-only",
  "admission_status": "proposed",
  "evaluation": {
    "semantic_suite_passed": null,
    "held_out_predictive_gain": null,
    "execution_cost": null
  }
}
```

A query definition can be exact relative to its declared interpreter while its inferred natural-language meaning remains uncertain. Store that uncertainty on the lexical/frame mappings and observation facts, rather than falsely weakening or strengthening the formal query definition.

## B Implementation tests and a minimal admission loop

### B.1 Tests that should run automatically

| Test                | Required behaviour                                                                                                                                                                                     |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Exact normalization | Applying the normalizer twice changes nothing. Renaming bound variables or reordering commutative input structure preserves the canonical result. Entity constants are not treated as bound variables. |
| Macro expansion     | Packed and expanded execution agree on Boolean results and required witness tuples. Overlapping matches do not create duplicate independent evidence.                                                  |
| Role reversal       | Swapping sender/recipient or agent/patient changes the relevant query answers while preserving unrelated facts.                                                                                        |
| Shared witness      | Replacing one repeated variable by a distinct witness can break a conjunction even when each component still matches somewhere.                                                                        |
| Scope preservation  | A denied, hypothetical, quoted, or believed event does not become an unqualified assertion. Unsupported quantifier constructions remain explicit.                                                      |
| Uncertain joins     | A union of individually plausible facts cannot satisfy a program unless at least one compatible hypothesis supports that joint match.                                                                  |
| Prefix invariance   | An unseen suffix, future memory insertion, or target-token change cannot alter the current observation record or features.                                                                             |
| Retraction          | Removing an observation invalidates dependent inferences and cached matches without deleting unrelated evidence.                                                                                       |
| Version safety      | A revised macro gets a new definition version; old programs either retain their semantics or undergo an explicit, tested migration.                                                                    |
| Predictive validity | Reader outputs and final forecasts are normalized; count updates occur after outcomes are observed; missing observations trigger defined backoff.                                                      |

## B.2 Admission-loop sketch

This pseudocode specifies sequencing, not a required programming language or training framework.

```
# Families, not individual paraphrases, define the splits.
D_mine, D_fit, D_admit, D_test = grouped_split(data)
freeze_manifest_and_evaluation_protocol()

bank = validated_initial_bank()
consumer = fit_consumer(bank, D_fit, fixed_budget)

for round_id in range(max_rounds):
    candidates = propose_from_existing_methods(bank, D_mine)
    candidates += propose_typed_binding_motifs(bank, D_mine)
    candidates = canonicalize_and_deduplicate(candidates)

    eligible = []
    for candidate in candidates:
        if not passes_static_and_semantic_tests(candidate):
            record_rejection(candidate, "semantic-contract")
            continue
        if not passes_prefix_availability_tests(candidate):
            record_rejection(candidate, "availability")
            continue
        eligible.append(candidate)

    baseline = matched_refit(bank, D_fit, fixed_budget)
    nominees = screen_with_training_residuals(eligible, baseline)
    trials = matched_candidate_refits(nominees, bank, D_fit)
    winner = select_on_admission_data(
        trials, baseline, D_admit,
        semantic_constraints, complexity_cost, runtime_cost
    )

    if winner is None:
        break
    bank = admit_with_provenance(bank, winner)
    consumer = refit_and_refresh_residuals(bank, D_fit)

# Search and thresholds are now frozen. Do not feed this result back.
report = evaluate_once_on_untouched_test(bank, consumer, D_test)
write_artifacts_and_failure_analysis(report)
```

The screen uses training information; the admission stage estimates generalization under a limited search budget; the final test is not part of the selection loop. In larger studies, rotate or nest admission folds to limit adaptive overfitting. For non-predictive packaging, replace the loss criterion with a declared cost-at-fixed-correctness criterion rather than pretending every useful macro must change a forecast.

## References

- [1] Ben Goertzel. *FUSE-NF: Constructing an Elegant Normal Form for Natural Language Semantics*. May 26, 2025. Supplied concept paper, 12 pages. Principal sections used: 3 (desired properties), 4.3 (five consolidation methods), 4.4–4.6 (application and enrichment), and 5–6 (integration and example).
- [2] Benjamin Goertzel. *NESS: NEural-Symbolic Sandwich. Reusable programs inside an adaptive predictor*. September 2026. Supplied technical briefing, 35 pages, including backup slides. File: `NESS_20min_Learning_and_Programs(1).pdf`. Principal material used: slides 2–6, 8–12, 19–23, B7, and B9. The briefing reports earlier experiments; this note does not independently reproduce them.
- [3] Benjamin Goertzel. *NESS, the Neural-Symbolic Sandwich: a new direction for making transformers AGI-friendly*. Supplied YouTube transcript printout, 29 pages, captured September 28, 2026. Used as architectural motivation, especially the description of semantic relation extraction and symbolic programs around 16:42–21:50. The more specific interface descriptions in this note follow the technical briefing in reference [2].
- [4] Developer progress update supplied by Benjamin Goertzel with the request. Undated message, received in the September 2026 discussion. Reports the approximate corpus size, ongoing verification of mining outputs, parser variability, and the proposed logical-template direction. Treated as a developer report rather than independently verified measurements.
- [5] `leungmanhin/semantic-parsing-hitl.fusenf/mining/out_tier_ab/patterns2_faithful.metta`. [GitHub artifact on the main branch](#), inspected September 30, 2026. Header and selected entries inspected; underlying miner and JSONL inventories not rerun.
- [6] `leungmanhin/semantic-parsing-hitl.fusenf/mining/out_tier_ab/rolefillers2_faithful.metta`. [GitHub artifact on the main branch](#), inspected September 30, 2026. Header and selected entries inspected; no claim of independent validation of proposed role mappings.
- [7] `leungmanhin/semantic-parsing-hitl.fusenf/mining/out_tier_ab/mi_faithful.metta`. [GitHub artifact on the main branch](#), inspected September 30, 2026. Header and selected entries inspected; reported thresholds and statistics not independently recomputed.
- [8] `leungmanhin/semantic-parsing-hitl.fusenf/mining/out_tier_ab/ae_faithful.metta`. [GitHub artifact supplied with the request](#). The inspected web preview did not expose the file contents. Recommendations about autoencoder analysis are based on the concept paper, not an audit of this export.

### Source and build note

All new objectives, interfaces, experiments, and milestones in this note are proposals. The supplied PDFs and limited repository inspection are distinguished from those proposals throughout. The LaTeX source is self-contained, uses ASCII input, and requires no external figures or bibliography database. Compile twice with `pdflatex` to resolve the contents and references.